Line Following Robot

Maitrey Amrutiya*, Eduardo Harari*, Keayen Jeppson*, Kevin Patel*, Nicholas Reichardt*, Javier Briceno*
* Multidisciplinary Engineering Technology, Department of Engineering Technology and Industrial Distribution,
Texas A&M University, TX 77843, USA

***Abstract*— The robot uses a three-sensor infrared (IR) array to detect line position and generate an error signal. The Arduino processes the input and executes a PD algorithm to minimize deviation from the path by modulating the PWM signals sent to two DC motors through a motor driver. The control parameters were designed and tuned in MATLAB/Simulink, where the system's transfer function was derived from experimental response data. Plot analysis verified system stability before implementation.**

Testing confirmed that the robot accurately follows curved and straight paths with minimal oscillation. Compared to on-off control, the PD approach provides smoother transitions, faster response, and improved stability. This project successfully integrates modeling, digital control design, and hardware implementation to demonstrate an efficient, standalone feedback-controlled robotic system.

Keywords*— *Line Following Robot, PD Control, Arduino, Embedded Systems, Feedback Control, PWM

I. Introduction

The automation of movement across the industrial and service sectors has needed the development of small, autonomous platforms capable of reliable navigation along predefined paths. This demand stems from the need for increased efficiency, reduced labor costs, and operational consistency in tasks like material transport, floor cleaning, and other applications. The nature of the problem lies in achieving this reliability and stability with an affordable, scalable system. Current low-cost autonomous vehicles, especially line-following robots, are often not very effective as they typically use simple on/off control logic based on minimal sensor input. These control methods result in poor line detection consistency, jerky motion, and a tendency to overshoot curves, making them unsuitable for use in environments requiring smooth and precise movements.

This project is motivated by the need to develop an affordable and reliable alternative that can serve as both a prototyping tool for small-scale automation and as an effective platform for teaching fundamental control system concepts. Our proposed solution is a closed-loop Proportional-Derivative (PD) line-following robot. This system utilizes an Arduino microcontroller to process real-time input from three front-mounted Infrared (IR) sensors, which are configured to generate a continuous, quantifiable error signal representing the robot's lateral deviation from the line. This error is fed into a discrete-time PD controller, which makes adjustments to the speeds of the two rear DC motors.

The commercial value and comfort of this solution stem from its ability to perform at a high level using common, low-cost hardware. Additionally, the system achieves smoother, more stable, and more consistent path tracking compared to more common open-loop systems. This allows a line-following system to be successfully used in varied environments that otherwise would fail with open-loop versions.

II. Prototype Description

The robot's core functionality is achieved through the integration of three systems: a sensing subsystem comprised of three infrared (IR) sensors for line position feedback; an actuation system consisting of two DC motors powered via an H-bridge motor driver for differential steering; a control subsystem centered on an Arduino microcontroller and a Proportional-Derivative (PD) control system, all powered by a 9V battery source.

A. Plant or Process

The plant in this control system is the motor controls. The plant receives the control signal, which is the requested motor speeds, and executes the motion, resulting in an output (the robot's actual position relative to the line). The plant introduces the dynamics that the controller manages, and the PD controller aims to manipulate the motor speeds to keep the plant centered on the line.

B. Actuators

The actuators are the devices that convert the electrical control signal from the Arduino into the physical action (motion) required to execute the PD control strategy. The primary actuators in this system are the DC motors, which convert electrical energy from the Arduino into mechanical energy. By driving the two independent back wheels, the motors provide the thrust and steering capability. To turn, the controller applies different wheel speeds or directions to the left and right motors, which are key characteristics of a differential drive. Additionally, while not directly an actuator, the H-bridge motor driver acted as an intermediary between the output of the Arduino and the DC motors.

C. Sensors

This system uses three IR sensors to provide a feedback signal to the controller. They function by emitting an infrared light beam and measuring the reflected light. A dark line (which absorbs IR) will result in a low reflection signal, while a bright floor (which reflects IR) will result in a high signal. The three sensors collectively determine the

robot's positional error. The middle sensor primarily checks if the line is directly beneath the robot, while the left and right sensors are used to detect when the robot drifts off the line. The sensor readings are sent back to the Arduino, which are used by the controller to calculate the error (how far the robot is from the center of the line), which is the input for the PD control system.

```
void loop() {
 int L = digitalRead(L_S);
 int C = digitalRead(C_S);
 int R = digitalRead(R_S);
 if (L == 1 && C == 1 && R == 1) {
   Stop();
   return;
 }
 if (L == 1 && C == 0 && R == 0) error = -1;  // drift left
 else if (R == 1 && C == 0 && L == 0) error = +1;  // drift right
 else error = 0;  // centered
 int P = Kp * error;
 int D = Kd * (error - lastError);
 int control = P + D;
 lastError = error;
 int leftPWM  = basePWM + control;
 int rightPWM = basePWM - control;
 leftPWM  = constrain(leftPWM, 60, 255);
 rightPWM = constrain(rightPWM, 60, 255);
 driveMotors(leftPWM, rightPWM);
}
void driveMotors(int leftPWM, int rightPWM) {
 digitalWrite(in1, LOW);
 digitalWrite(in2, HIGH);
 digitalWrite(in3, HIGH);
 digitalWrite(in4, LOW);
 analogWrite(enA, rightPWM);
 analogWrite(enB, leftPWM);
}
```

Figure 1: Main Program Logic

III. System Model

A. System Structure and Modeling Approach

The system is modeled on a PD feedback control system for which the plant consists of the robot's steering, the controller is based on a Proportional Derivative (PD), and the error signal is generated by the lateral IR sensors. The system is modeled on the input u(t), error e(t), and output y(t). The system takes the error calculated from the lateral IR sensors, passes it onto the controller, and outputs a modified PWM signal to the respective motor for line correction.

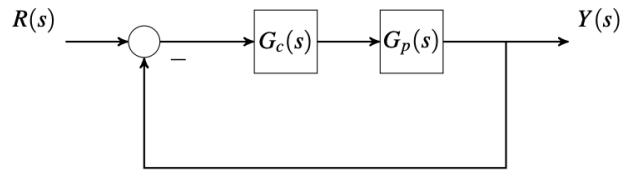

Figure 2: System Block Diagram

B. Units

Variable	Description	Units
e(t)	IR Sensor error	Sensor(-1/0/+1)
y(t)	output	PWM (0-255)
k	Gain	Unitless
τ	Time Constant	Millisecond (ms)
ω_n	Natural Frequency	Rad/s
$2\zeta_n$	Damping Ratio	Unitless
K_p	Proportional gain	Unitless
K_d	Derivative Gain	Milliseconds (ms)

Table 1: Unit Description

C. Equations

$$G(s) = \frac{k}{\tau s+1} * \frac{1}{s}$$

Equation 1: Plant Transfer Function

$$C(s) = K_p + K_d s$$

Equation 2: Controller Transfer Function

$$G_{CL}(s) = \frac{C(s)G(s)}{1+C(s)G(s)}$$

Equation 3: Closed Loop Transfer Function

$$G_{CL}(s) = \frac{(K_p+K_d s)(\frac{k}{\tau})}{s^2+(\frac{K_d k+1}{\tau})s+\frac{K_p k}{\tau}}$$

Equation 4: Completed Closed Loop Transfer Function

$$s^2 + 2\zeta_n\omega_n s + \omega_n^2$$

Equation 5: Second-order transfer function form

$$K_p = \frac{\omega_n^{\ 2}}{10.75}$$

Equation 6: Derived Kp Equation

$$K_d = \frac{2\tau\zeta_n\omega_n - 1}{k}$$

Equation 7: Derived Kd Equation

IV. Controller Design

The controller design is centered around a closed loop system capable of interpreting sensor feedback and generating the necessary motor PWM commands. Because the robot's motion is dependent on the response time and signal smoothness to its lateral position, the design process took time in translating the physical robot's behavior into a mathematical model and correlating that with a control strategy appropriate for execution. Using this experimental model, a Proportional Derivative (PD) controller was chosen to balance response time and stability of the system, while also keeping oscillations and overshoot relatively low. The following subsections outline the Performance Criteria, Controller structure, stability considerations, and implementation time throughout the entire controller design.

A. Performance Criteria

The performance objectives for this line following robot were defined to ensure the robot was stable, smooth, accurate, and repeatable during the path tracking process in real time. The primary goal of the control system is to minimize the lateral position error between the robot and the center of the line while maintaining continuous forward motion. The robot required a response quick enough to ensure it corrected its error, but also controlled enough to not have excessive oscillation or overshooting. Additionally, the controller must maintain steady tracking behavior through curves and slight turns without losing its track or exhibiting jerky motion.

B. Closed-Loop Controller

The closed-loop controller for this robot was implemented using a Proportional-Derivative (PD) controller. The controller continuously processes the error signal generated from the three IR sensors located at the front of the robot. This error represents the lateral displacement between the robot's centerline and the centerline of the path. The proportional gain provides immediate corrective action based on how large the error is, which allows the robot to respond accordingly to the deviations. The derivative improves the system's damping by reacting to the rate of change of the error, reducing overshoot and oscillations. The output of the PD controller directly adjusts the PWM signals sent to each wheel's motors through the motor driver, allowing for real-time feedback and correction.

C. Stability Analysis

The system's stability was evaluated by analyzing the closed-loop transfer function and comparing it to the standard second-order system to identify the equations relating the chosen damping and natural frequency to the Kp and Kd values. The chosen values were selected to ensure the system was stable and well-damped during its runs. Experimental testing confirmed that the controller prevented excessive oscillations and maintained smooth tracking during each test track. The stability analysis verified that the selected controller gains keep the system stable and efficient during its operations.

D. Controller Timing

The control was executed in the Arduino Uno R3 microcontroller. During each iteration of the loop, the sensor readings were acquired, the error was then computed, and the PD control applied updated PWM motor speeds to both motors. While the controller is implemented digitally, the design is based on a continuous-time control model that was developed using experimental characterization. The Arduino executed the control logic at a high rate to ensure smooth response time, which allowed the robot to react effectively to rapid changes.

V. Controller Implementation

This control system was implemented on an Arduino Uno R3. The first step that was needed to implement the controller was to first create a way to use the error values that were obtained from the sensors. Since they were digital, the error was calculated to go between 1 and -1 in increments of 0.5, with the left sensor set to -1, center to 0, and right to 1. This error was then given to the controller. The controller was programmed following the transfer function that was obtained, along with the calculated values. This created a controller that changed the speed of the wheels according to how far from the line the robot was.

VI. Controller Evaluation

This controller was tested through the means of repeated physical tests, which allowed the accounting of all possible side effects. Through this testing, it was determined that one motor was more efficient than the other, and an appropriate gain was then applied, limiting that motor. The controller was then tuned to find the best values by testing the sets of values given until the best solution was found. This led to the finalization of the Kp and Kd values as 11.977 and 1.783, respectively. The critical evaluation of the controller showed that it was a stable system, as all of the roots of the characteristic equation were negative. After the implementation of the controller, the stability was confirmed as the controller kept the robot moving along the line and not letting it veer off. However, due to the nature of the sensors, there was some jerky motion that was minimized through further testing and alterations. Additionally, there was no overshoot as the robot never turned far enough to trigger the other sensor, and there was no oscillation. It also had a response time of ~120-160 ms, which was fast enough to create an accurate and precise response. This control was evaluated against the controller design performance criteria that were previously mentioned. The final result was that of the robot, which was able to exhibit the behavior described in the controller design performance criteria to the best of its ability due to the hardware that was used.

VII. Discussion

Many of the topics covered in class were applied throughout this project, and the overall progress depended heavily on the concepts introduced during the course. A central topic utilized during development was the creation and evaluation of transfer functions and controllers. These concepts formed the foundation of the control strategy for the line-following robot.

The robot's operation relies on its system transfer function in combination with a PD controller, which together enable accurate and smoother motion along the line. Knowledge of system modeling, stability analysis, controller tuning, and performance evaluation was essential for determining appropriate gain values and ensuring stable, responsive behavior.

The experience that was gained through the completion of this project is an expansion of the knowledge gained throughout this course. This led to an experience in implementing control systems in real applications, in addition to how to model a real system to create a control system.

References

[1] ZRob314, "Line Following Robot with Basic PD (proportional-derivative) Control," Instructables, https://www.instructables.com/Basic-PD-Proportional-Derivative-Line-Follower/ (accessed Dec. 8, 2025).

[2] ZRob314, "Line Following Robot with Basic PD (proportional-derivative) Control," Instructables, https://www.instructables.com/Basic-PD-Proportional-Derivative-Line-Follower/ (accessed Dec. 8, 2025).

Appendix

```
#define enA 3
#define in1 6
#define in2 7
#define enB 9
#define in3 10
#define in4 11
#define L_S 2
#define C_S 4
#define R_S 5
float Kp = 12.0;
float Kd = 1.8;
int error = 0;
int lastError = 0;
int basePWM = 150;
void setup() {
 Serial.begin(9600);
 pinMode(L_S, INPUT);
 pinMode(C_S, INPUT);
 pinMode(R_S, INPUT);
 pinMode(enA, OUTPUT);
 pinMode(enB, OUTPUT);
 pinMode(in1, OUTPUT);
 pinMode(in2, OUTPUT);
 pinMode(in3, OUTPUT);
 pinMode(in4, OUTPUT);
 digitalWrite(in1, LOW);
 digitalWrite(in2, HIGH);
 digitalWrite(in3, HIGH);
 digitalWrite(in4, LOW);
}
void loop() {
 int L = digitalRead(L_S);
 int C = digitalRead(C_S);
 int R = digitalRead(R_S);
 if (L == 1 && C == 1 && R == 1) {
   Stop();
   return;
 }
 if (L == 1 && C == 0 && R == 0) error = -1;
 else if (R == 1 && C == 0 && L == 0) error
= +1;
 else error = 0;
 int P = Kp * error;
 int D = Kd * (error - lastError);
 int control = P + D;
 lastError = error;
 int leftPWM  = basePWM + control;
 int rightPWM = basePWM - control;
 leftPWM  = constrain(leftPWM, 60, 255);
 rightPWM = constrain(rightPWM, 60, 255);
 driveMotors(leftPWM, rightPWM);
}
void driveMotors(int leftPWM, int rightPWM)
{
 digitalWrite(in1, LOW);
 digitalWrite(in2, HIGH);
 digitalWrite(in3, HIGH);
 digitalWrite(in4, LOW);
 analogWrite(enA, rightPWM);
 analogWrite(enB, leftPWM);
}
void Stop() {
 analogWrite(enA, 0);
 analogWrite(enB, 0);
 digitalWrite(in1, LOW);
 digitalWrite(in2, LOW);
 digitalWrite(in3, LOW);
 digitalWrite(in4, LOW);
}
```

Figure 3: Full Program Code

Zeta	Wn (rad/sec)	Kp	Kd
0.4	10.1	9.484394703	0.1701445182
0.6	10.4	10.05619185	0.5792351632
0.8	10.7	10.64472453	1.010639843
1	11	11.24999274	1.464358559
1.1	11.3	11.87199647	1.730267478
1.12	11.35	11.97729082	1.782705461
1.2	11.4	12.08305005	1.962705344

Table 2: Tested Iterations for Zn & Wn

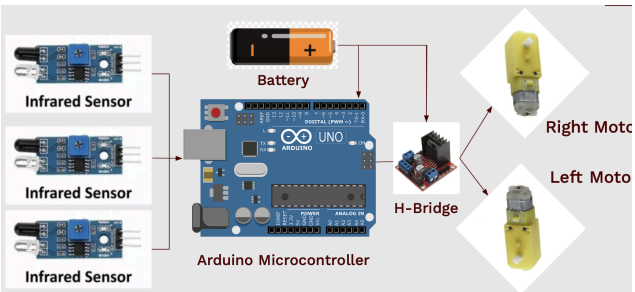

Figure 4: Functional Block Diagram of System

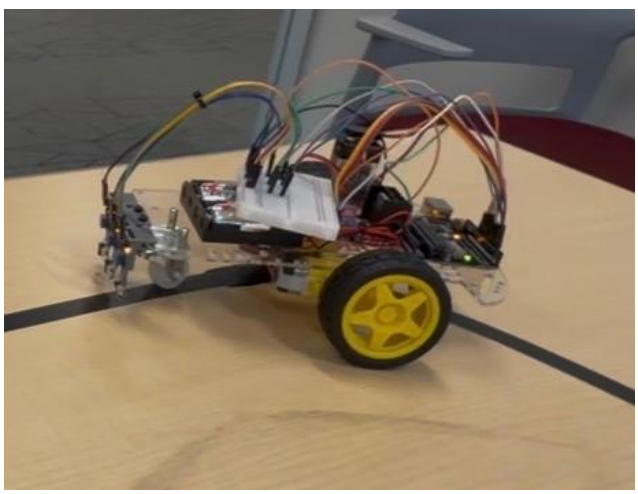

Figure 5: Line Following Robot - Physical System